GSEA-Profile: Characterizing and Optimizing Tool Orchestration Subsystems in Agentic Financial Analysts for Frontier Equity Markets

Bennet Brown(bnb2122)

May 6, 2026

1 Application and Design Pattern

1.1 Problem Description

Frontier financial environments like the Ghana Stock Exchange (GSE) present significant information processing barriers for local and international investors. Financial reports are often distributed as unstructured PDF files. These documents are a mix of clean, programmatically generated text and legacy, non-programmatic scanned document images.

To evaluate a company's financial health, human analysts must manually extract tabular data from financial statements and synthesize qualitative executive commentary. This introduces operational friction and delays in trading environments where time-sensitive decision-making is critical.

2 Agentic Solution

To automate this task, this project introduces GSEA-Agent (Ghana Stock Exchange Analyst), an agentic system built using open-source workflows. The agent maps unstructured disclosures into structured equity summaries and forward-looking stock outlook predictions. We implement a Planning and Decomposition design pattern to break down the analytical workflow into isolated, manageable components:

Macro-Planner: Ingests a top-level user prompt (e.g., "Provide a financial health profile and market outlook for Dannex Ayrton Starwin PLC based on their latest Q1 unaudited statement") and decomposes it into sequential subtasks.

Perception & Tabular Inversion Layer: A sub-agent or tool equipped with layout-aware vision models converts unstructured text and scanned document tables into clean, structured data.

Synthesis Engine: An investigative sub-agent reads the structured tables alongside qualitative narrative logs, computes structural financial changes, evaluates management risk strategies, and compiles the final report payload.

3 Execution Plan

3.1 Infrastructure Topology

The agent runtime is implemented using open-weights models hosted on a Cloud TPU v5e VM equipped with a serving layer that is powered by the vLLM optimization engine.

3.2 Experimental Variations

The core of our systems characterization focuses on the infrastructure tradeoffs between two fundamentally different tool orchestration mechanisms used to provide context to the Synthesis Engine:

Mechanism A (Na ïve Long-Context Prefill): The agent loads the entire extracted document library directly into the active prompt window for a massive single-pass evaluation.

Mechanism B (Directive RAG Tool Orchestration): Documents are sorted and converted to vector embeddings. The sub-agent treats the database as a tool, dynamically querying the index to selectively retrieve only the specific context blocks required for immediate sub-questions.

Our primary implementation will be mechanism A, with implementation of mechanism B as stretch goal.

3.3 Emperical Profiling Metrics

Empirical Profiling Metrics To evaluate application efficiency against hardware costs, we profile four specific metrics:

Time to First Token (TTFT): The latency penalty encountered during initial prompt evaluation.

Inter-Token Latency (ITL): The generation fluidity (seconds per token) during autoregressive decoding.

End-to-End Latency: The total wall-clock time required to execute the complete decomposition loop.

HBM Activation Memory Footprint: The transient memory capacity consumed on the TPU chip.

4 Hypotheses

Our systems benchmarking is directed by two contrasting hypotheses regarding the primary computational constraints of each tool orchestration method:

Hypothesis I: The Long-Context Prefill Bottleneck (Mechanism A)
When using Mechanism A (Long-Context Prefill), the primary infrastructure constraint is the quadratic self-attention bottleneck ($O(N^2)$) during the initial prompt processing pass.

Hypothesis II: The RAG Tool-Use Orchestration Gap (Mechanism B)
When executing queries via Mechanism B (Dynamic RAG Tool Use), the runtime characteristics shift away from raw tensor computation to tool-use orchestration downtime. Every time the agent interrupts generation to invoke the retrieval index, the inference system must pause token decoding, yield execution control back to the hosting Python runtime, and wait for external computation to complete. This creates a "latency bubble" that reduces overall hardware utilization.